Municipal Budget Allocation as a Multi-Dimensional Knapsack

Exact (CP-SAT) versus heuristic (Genetic Algorithm) optimization on real participatory-budgeting data from Warszawa 2023

Group members	Student ID	Contribution (summary)
S. Thanushan	MS26912684	Data pipeline, constraint generation, greedy baseline, mathematical formulation, report and video assembly
G.C. Devinda	MS26902616	Genetic Algorithm, exact CP-SAT model, comparative evaluation, report and video assembly

Problem category	Resource Allocation (Budget)
Dataset	Pabulib, Warszawa 2023 (138 projects, 30,339,246 PLN) (https://pabulib.org/visualize/Poland_Warszawa_2023.pb)
Methods	Exact: CP-SAT (OR-Tools) · Heuristic: Genetic Algorithm · Baseline: value/cost greedy
Repository	https://github.com/chirangovinna/IT5082-OM-Assignment
Video	https://www.youtube.com/xxx

1. Problem Selection and Motivation

Public authorities regularly face the question "which of these proposals should we pay for?". In *participatory budgeting* (PB) the proposals come from residents, residents vote on them, and the city funds winners until the money is gone. The decision is a resource-allocation problem with three features that make it a good optimization case: (i) the demand for money far exceeds supply, so the constraint is tight; (ii) the objective, public support, is measurable; and (iii) real policies add side rules, such as not letting one policy area absorb everything. We chose it for four reasons.

- **It is realistic.** The data are an actual 2023 municipal election with real costs, real votes and a real decision that we can compare against.
- **It is non-trivial.** With 138 binary decisions there are $2^{138} \approx 3.5 \cdot 10^{41}$ candidate portfolios enumeration is out of the question. The underlying 0/1 knapsack problem is NP-hard.
- **It supports a fair exact-versus-heuristic comparison.** It is small enough for an exact solver to give a provable optimum, so the quality of any heuristic can be measured exactly rather than guessed.
- **It is extensible.** Dependencies, minimum quotas or multi-year budgets can be added to the same model.

The assignment brief lists Resource Allocation (Budget) as an accepted problem category, so no instructor approval was required.

2. Problem and Data Description

2.1 Decision problem

Given a total budget B and a set of n candidate projects, each with a cost and a measure of community support, choose the subset of projects to fund that maximises total support, without exceeding the budget and without exceeding a spending cap on any project category. Projects are indivisible: a project is either fully funded or not funded.

2.2 Dataset

We use the **Warszawa 2023** instance of the Pabulib participatory-budgeting library (Faliszewski et al., 2023), file `Poland_Warszawa_2023.pb`, a publicly available, citable collection of real PB elections. The file has three sections: META (budget, number of voters, vote type), PROJECTS (cost, votes, category, whether the city selected it) and VOTES (individual ballots). Following the assignment's principle, the data are used only to instantiate and validate the optimization model; nothing is trained.

Property	Value
Source election	Warszawa (Poland), edition 9, voting 15–30 June 2022, approval voting, city rule "greedy"
Candidate projects	138
Total budget B	30,339,246 PLN
Total cost of all candidates	175,021,441 PLN (5.8 × the budget)
Project cost	min 2,800 · median 385,000 · mean 1,268,271 · max 6,067,700 PLN
Votes per project	min 108 · median 2,728 · mean 4,703 · max 27,805 (total 648,967)
Categories (primary label)	10: urban greenery, sport, public space, welfare, environmental protection, education, health, culture, public transit and roads, uncategorized
City's actual selection	22 projects, 30,336,684 PLN (99.99% of budget), 290,845 votes

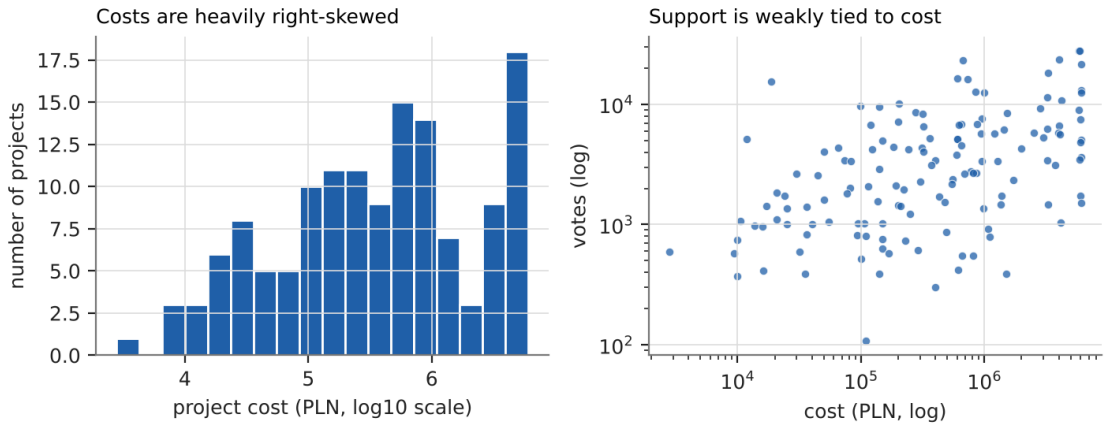


Figure 1. Distribution of project costs (left) and relationship between cost and votes (right) in the Warszawa 2023 instance.

Costs span more than three orders of magnitude, and a few large projects carry much of the cost.

2.3 How the data map to the model

Model symbol	Pabulib field	Meaning
c_i	cost	cost of project *i* in PLN
v_i	votes	number of approval votes, used as the value of funding project *i*
B	META: budget	total budget
P_k	category	set of projects in category *k*
(validation only)	selected	the city's real outcome, used as an external benchmark

2.4 Constraints and their justification

Budget. Total spending cannot exceed B. **Category caps.** Without further rules, an optimizer simply funds the projects with the best votes per złoty and can pour most of the budget into one or two areas. Real municipal policy rarely allows that. We therefore cap each category's spending in proportion to that category's share of all votes, with 25% slack ($\sigma = 1.25$), and never above the category's own total cost. The cap formalises the rule "spend roughly in line with demonstrated demand". The slack value is a modelling choice, not a legal rule of the city; it is the main assumption of this study and we return to it in Section 7.3.

2.5 Instances of different size

To study scalability we drew random subsets of the 138 projects (15, 30 and 60 projects, random seeds 42, 43 and 44). For each subset the budget was set to the same fraction of the subset's total cost as in the full instance (17.3%), so every instance is equally selective, and the category caps were recomputed with the same formula. The full instance uses the real budget.

Instance	Projects	Budget (PLN)	Total candidate cost (PLN)	Total votes
small	15	2,567,686	14,812,500	54,074
medium	30	8,570,663	49,442,555	166,555
large	60	13,210,421	76,208,451	250,316
full (real)	138	30,339,246	175,021,441	648,967

3. Mathematical Formulation

The problem is a 0/1 multi-dimensional knapsack with one global resource constraint (the budget) and one resource constraint per category. All symbols are defined in the table; the model follows.

Symbol	Type	Definition
n	parameter	number of candidate projects
$i \in \{1,\dots,n\}$	index	project
K	set	set of categories
$P_k \subseteq \{1,\dots,n\}$	set	projects in category $k \in K$; the P_k partition the projects
c_i	parameter	cost of project i (PLN, integer)
v_i	parameter	value of project i = its approval votes (integer)
B	parameter	total budget (PLN)
B_k	parameter	spending cap of category k (defined below)
σ	parameter	slack factor, 1.25
x_i	decision variable	1 if project i is funded, 0 otherwise

Objective: maximise the total votes of the funded projects.

$$\max \quad Z = \sum_{i=1}^{n} v_i x_i \qquad \text{(1) objective}$$

Subject to:

$$\sum_{i=1}^{n} c_i x_i \le B \qquad \text{(2) overall budget}$$

$$\sum_{i \in P_k} c_i x_i \le B_k \qquad \forall k \in K \qquad \text{(3) category caps}$$

$$x_i \in \{0,1\} \qquad \forall i \in \{1,\dots,n\} \qquad \text{(4) binary decisions}$$

Category caps are computed from the data as

$$B_k = \min\left(\sigma B \frac{\sum_{i \in P_k} v_i}{\sum_{i=1}^{n} v_i},\ \sum_{i \in P_k} c_i \right), \qquad \sigma = 1.25 \qquad \text{(5) cap rule}$$

The model has n binary variables and $1 + |K|$ linear constraints. It is feasible by construction ($x = 0$ satisfies every constraint). Constraint (3) is not redundant: the caps sum to 125% of the budget, so they can bind, and two of the ten caps do. Because the P_k partition the projects, the constraints have a block structure explains why this matters for solver difficulty.

4. Optimization Methods

The brief requires one exact method and one heuristic or metaheuristic. We implemented those two and added a third, simple baseline so that the value of the more elaborate heuristic can be judged against something cheap. All three solve exactly the same model and are checked for feasibility against constraints (2) to (4).

4.1 Exact method: CP-SAT (Google OR-Tools)

What it is. CP-SAT is a constraint-programming solver built on SAT-style clause learning, with linear relaxations and cutting planes. Given the objective and linear constraints it searches the space of 0/1 assignments, prunes with propagation and bounds, and terminates with a proof of optimality or a bound on the gap.

Why we chose it.

(i) It handles integer linear models directly, so constraints (2) to (4) are written almost verbatim.

(ii) It returns a proof of optimality, which turns it into ground truth for measuring heuristics.

(iii) It needs no hand-written branching code, and is more robust than a plain LP-based branch-and-bound on models with many binary variables.

4.2 Heuristic: Genetic Algorithm

What it is. A population-based metaheuristic that evolves candidate portfolios.

Why we chose it. The natural encoding of a knapsack solution is a bit string, which fits the GA directly, and the GA is among the techniques the brief names. We wrote it from scratch (NumPy only) so each operator can be explained and tested.

Design.

Component	Choice
Chromosome	binary vector of length n; bit i = 1 means project i is funded
Fitness	total votes of the funded projects
Initial population	100 random bit strings, each repaired to feasibility
Selection	tournament of size 3
Crossover	single point, probability 0.8
Mutation	bit flip, probability 0.02 per bit
Elitism	best 2 individuals copied unchanged
Termination	200 generations
Repair operator	while any constraint is violated, take the largest violation (budget or one category) and remove the funded project with the lowest votes per PLN inside it; repeat until feasible

The repair operator is the key design decision. Instead of penalising infeasible individuals, which wastes search effort in infeasible regions, every individual is made feasible immediately, so the whole search takes place inside the feasible set. Its cost is a bias towards solutions that look like the greedy ratio rule, which may limit the heuristic's ability to escape it. Because the GA is stochastic, we report it over 10 random seeds (0 to 9) and not from one run.

4.3 Baseline: greedy value-per-cost

Sort the projects by votes per PLN and fund them in that order, skipping any project that would break the budget or its category cap. It runs in $O(n \log n)$ time, has no randomness and never backtracks. For the single-constraint knapsack it is a well-known strong heuristic, which makes it a demanding baseline for the GA.

4.4 NP-hardness and why two kinds of method are needed

The 0/1 knapsack problem is NP-hard, so no polynomial-time exact algorithm is known for the general case, and the multi-constraint version is at least as hard. Exact solvers can therefore blow up on some instances, which motivates heuristics that trade a guarantee for speed and generality. Whether that trade is worthwhile on a given instance is an empirical question, and answering it is the purpose of the experiments below.

5. Implementation Summary

The pipeline is written in Python 3 (pandas, NumPy, OR-Tools, Matplotlib) as small modules with a single responsibility each, plus a harness that runs all methods on all instances. All paths resolve relative to the project root, so the scripts run from any working directory, and all random seeds are fixed, so every table in this report can be regenerated.

Module	Responsibility
pb_parser.py	reads a Pabulib .pb file into a metadata dictionary and pandas tables
data_prep.py	builds the four instances and their budgets
constraints.py	computes category caps with rule (5)
exact_cpsat.py	CP-SAT model of equations (1) to (4)
heuristic_ga.py	Genetic Algorithm with constraint-aware repair
greedy_baseline.py	value-per-cost greedy with category caps
comparison.py	runs all methods, writes tables and plots

Verification. Every reported solution was checked for feasibility, meaning budget respected and every category cap respected, and CP-SAT's status was OPTIMAL on all eight model runs reported in Section 6 (four instances, with and without caps). Parser output was checked against the file header (138 projects, budget 30,339,246) and the city's selection (22 projects, 30,336,684 PLN).

6. Results

All runs used a standard cloud virtual machine, so absolute times are indicative ratios and orderings are what matter. "Gap" is optimum, where the optimum is the proven CP-SAT value. The CP-SAT and greedy times are medians of 15 repeated runs; GA figures are means over 10 seeds.

6.1 Solution quality and runtime on the capped model (main experiment)

Size	*n*	Optimum (votes)	Greedy gap	GA mean gap (10 seeds)	GA seeds reaching optimum	CP-SAT time (s)	Greedy time (s)	GA time (s)
small	15	26,242	0%	0%	10/10	0.003	0.001	1.13
medium	30	57,590	0%	0%	10/10	0.003	0.002	1.80
large	60	166,567	0%	0%	10/10	0.008	0.003	2.22
full	138	371,661	0.81%	1.01% (0.07–1.82%)	0/10	0.108	0.009	3.72

Table 1. Main results on the capped model. CP-SAT proved optimality in every case.

On the three smaller instances all three methods find the optimum. The differences appear only on the full instance, where the greedy baseline lies 0.81% (3,027 votes) below the optimum, and the GA between 0.07% and 1.82% below it, with a mean of 1.01%. No GA seed found the optimum on the full instance. With seed 42, the run shown in the other tables and figures, the GA reaches 368,787 votes (gap 0.77%), selecting 76 projects against 82 for CP-SAT and 91 for the greedy rule.

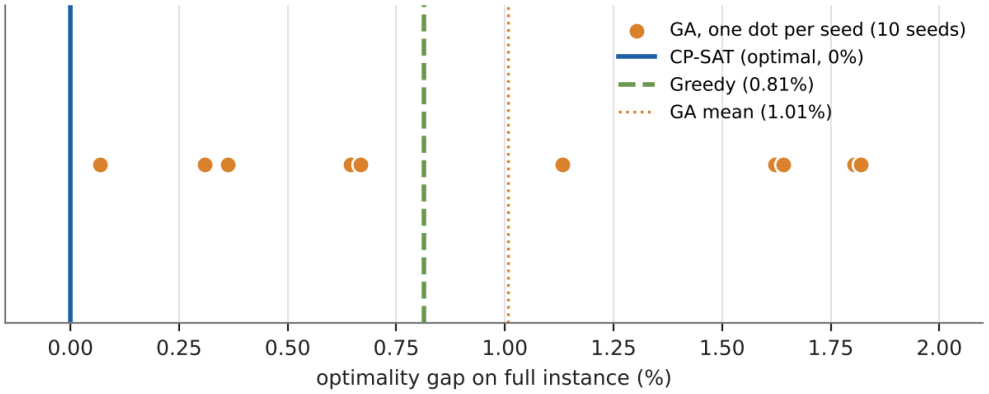


Figure 2. Optimality gap on the full instance. CP-SAT is optimal by definition.

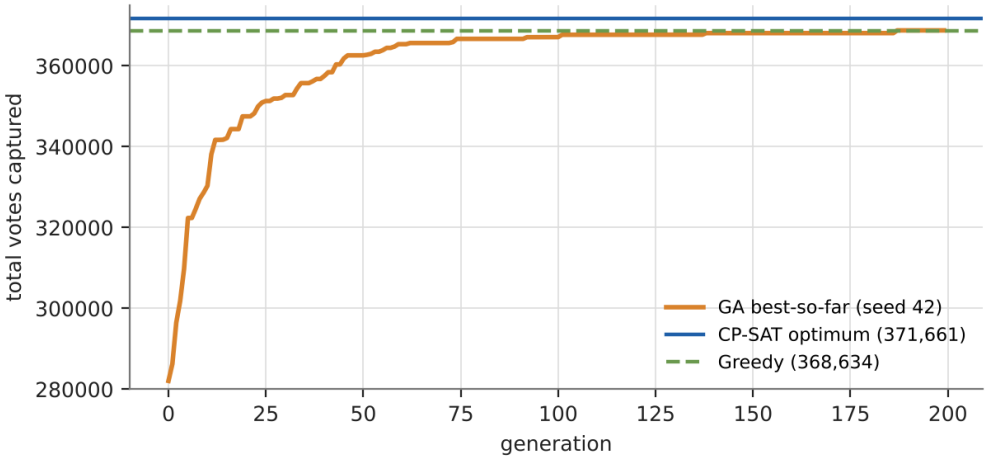


Figure 3. Convergence of the GA (seed 42) on the full instance. It rises quickly, then improves slowly in the last generations without reaching the CP-SAT value or even the greedy value.

6.2 Runtime and scalability

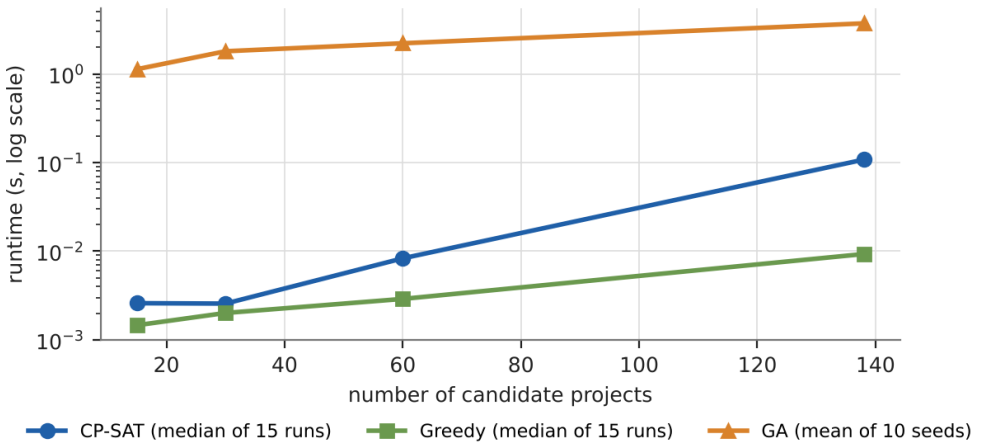


Figure 4. Runtime against instance size (log scale). The ordering greedy < CP-SAT < GA holds at every size, and CP-SAT is more than 30 times faster than the GA at 138 projects.

CP-SAT's time grows with size (0.003 s at 15 projects, 0.108 s at 138) but remains far below a second. The GA's time grows more slowly in relative terms, but it starts from a high base (a population of 100 over 200 generations with a repair step in Python) and stays roughly 30 to 450 times slower than CP-SAT at every size. We stress that all instances here are small by optimization standards, so these timings say nothing about much larger problems.

6.3 Effect of the category caps

Instance	CP-SAT time, no caps (s)	CP-SAT time, with caps (s)	Greedy gap, no caps	Greedy gap, with caps	GA gap, no caps (seed 42)	GA gap, with caps (seed 42)
small	0.006	0.003	0%	0%	2.21%	0%
medium	0.006	0.003	0%	0%	0%	0%
large	0.017	0.008	0.12%	0%	0%	0%
full	0.033	0.108	0%	0.81%	0.57%	0.77%

Table 2. Single-constraint model (budget only) versus capped model. Times are medians of 15 runs; heuristic gaps are for one run each (GA seed 42).

Adding the caps made the problem measurably harder only for the full instance, where CP-SAT's median time rose by a factor of about 3.3 (0.033 s to 0.108 s) and the greedy baseline changed from optimal to 0.81% below optimal. On the smaller instances the caps made no difference to quality, and CP-SAT was actually faster with them, plausibly because the extra constraints shrink the search space. We therefore do *not* claim that caps make the problem uniformly harder; their effect is instance-dependent and modest.

6.4 Comparison with the city's actual selection

Selection	Projects	Cost (PLN)	Votes captured	Overlap with city's 22
City's actual (greedy-by-votes rule)	22	30,336,684	290,845	22
CP-SAT optimum (with caps)	82	30,300,476	371,661 (+27.8%)	18
GA (seed 42)	76	30,214,676	368,787 (+26.8%)	18

Table 3. Comparison with the real outcome on the full instance.

Under the same objective, total votes, an optimization approach funds many more, smaller projects and captures 27.8% more votes with practically the same money. The comparison has limits: the city's rule ranks projects by votes only, with no regard for cost, and may be subject to rules that our model does not contain; the cap rule (5) also restricts the optimum while the city's rule was not subject to it. Accordingly, the result shows that maximising total approvals under a budget is not what a rank-by-votes rule achieves, not that the city chose badly, because the city was probably optimizing for something other than total approvals, such as the visibility of winning projects.

6.5 Which constraints are active

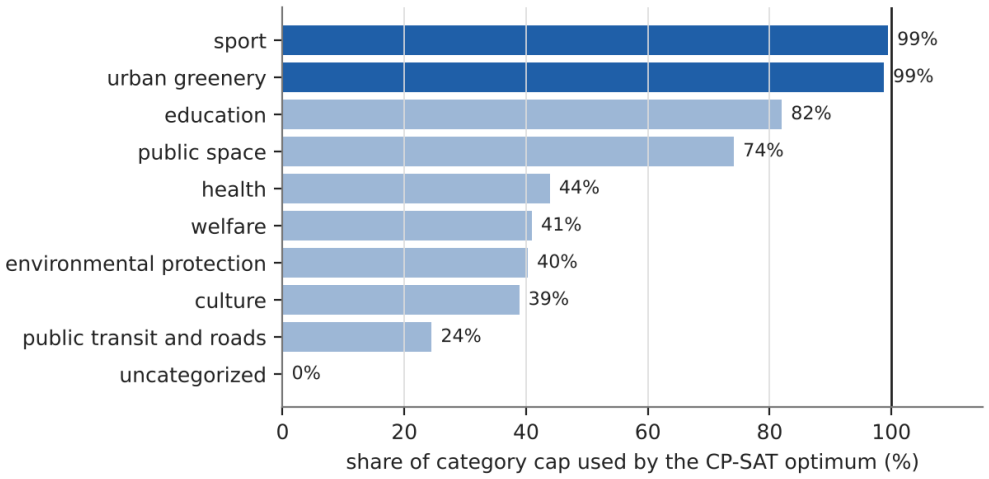


Figure 5. Share of each category cap used by the CP-SAT optimum on the full instance.

Only two of the ten caps are binding. The optimum spends 13.00 M PLN on urban greenery and 7.75 M on sport, each at 99% of its cap. The others are far from binding, and one category ("uncategorized") receives nothing. This explains why the capped problem differs only modestly from the budget-only problem.

7. Discussion

7.1 Why the exact method is so fast

The headline result is that CP-SAT solves a general-purpose NP-hard problem to proven optimality in about a tenth of a second at 138 projects. NP-hardness is a worst-case statement, and our instances are far from worst case. Three factors help. First, the number of variables is small. Second, the structure is plausibly benign: the linear relaxation (allow fractional funding) is probably very close to the integer optimum, which would make bounding effective (we did not measure the relaxation gap). Third, the category constraints partition the projects, so the model is a set of ordinary knapsacks coupled by one budget. Such a structure is also solvable in pseudo-polynomial time by dynamic programming, which is consistent with, though not a proof of, the solver's ease. We have not tested this explanation directly, for example by timing instances with strongly correlated costs and values, which are known to be hard for knapsack solvers.

7.2 What the heuristics show

The greedy rule is a strong baseline: it is optimal on three of four capped instances and within 0.81% on the fourth, in under 10 ms. The GA is not better: it is slower than greedy by two to three orders of magnitude, and on the full instance worse than greedy on average. A likely cause is the repair operator, which removes the lowest-ratio projects first and so pulls every offspring towards greedy-like solutions, while the GA's search is not guided enough to improve on them in 200 generations. We have not run an ablation (for example a penalty-based GA, or larger populations) to confirm this, so it remains a hypothesis. The spread across seeds, 0.07% to 1.82%, also shows that a single GA run is not a reliable summary; a one-seed comparison could have made the GA look either clearly better or clearly worse than greedy.

7.3 Trade-offs and when to use each method

	CP-SAT (exact)	Genetic Algorithm	Greedy
Quality	provably optimal	0.07–1.82% below optimum on 138 projects; optimal on smaller ones	0.81% below optimum on 138 projects; optimal on smaller ones
Speed (138 projects)	0.11 s	3.7 s	0.009 s
Guarantee	optimality proof	none; varies with seed	none
Flexibility	needs a linear model	any fitness and constraint that can be checked	needs a sensible ranking rule
Best used when	the model is linear and of moderate size	the problem is non-linear or too large or irregular for exact solvers	a fast, explainable answer is enough

For this problem as modelled, CP-SAT dominates: it is the most accurate and, apart from the trivial greedy rule, the fastest. A GA would earn its place only if the problem acquired features that a linear solver cannot express, or grew beyond what an exact solver can handle.

Limitations

- **Cap rule is our assumption.** The caps ($\sigma = 1.25$, proportional to vote share) are a plausible policy, not the city's. With a different σ the number of binding caps and the gaps would change. We did not run a sensitivity analysis on σ.
- **Votes as value.** Approval counts ignore overlap between voters and do not weigh fairness between districts or groups; maximising the sum of approvals can favour large majorities.
- **Primary category only.** Many projects carry several category labels; we use only the first, which simplifies the caps and misallocates some cost.
- **Small and benign instances.** The largest instance has 138 variables and the structure is easy; the conclusions about runtime do not transfer to harder or larger problems.

- **Smaller instances are random subsets** with a rescaled budget, drawn once each; results at those sizes come from one draw per size.
- **GA not tuned.** Hyper-parameters were set once and not optimised, and timings come from an unoptimised Python implementation, so the GA's runtime disadvantage is partly an implementation effect.

7.4 Future work

- Sensitivity analysis on the slack σ and on alternative cap rules, such as equal caps or caps taken from the city's rules.
- Additional constraints: project dependencies ($x_B \leq x_A$), minimum numbers of projects per category, and district fairness; these destroy the partition structure and may make exact solving harder, which is the regime in which heuristics should matter.
- Harder instances: strongly correlated costs and values, or synthetic instances with thousands of projects, to locate the point at which CP-SAT slows down and the GA becomes competitive.
- Stronger heuristics: simulated annealing, a GA with a penalty-based fitness or a local-search step (memetic), and ablations of the repair operator.
- Other objectives: proportionality-based voting rules such as the Method of Equal Shares, compared with utilitarian welfare maximisation on the same data.
- Multi-year budgets with carry-over of unspent money.

8. Individual Contributions

The work was split along the pipeline so that each member owns identifiable components and commits in the repository. Both members took part in choosing the problem, interpreting the results and preparing for the viva.

Component	MS26912684	MS26902616
Problem selection, dataset choice	shared	shared
Parser, instance preparation, constraints module	●	
Mathematical formulation and symbol table	●	
Exact CP-SAT model		●
Genetic Algorithm and repair operator		●
Greedy baseline	●	
Comparison harness, figures, supplementary experiments		●
Report	●	●
Video recording and repository organisation	shared	shared

9. Conclusion

We modelled municipal budget allocation as a 0/1 multi-dimensional knapsack, instantiated it with the real Warszawa 2023 participatory-budgeting data, and compared an exact solver, a Genetic Algorithm and a greedy baseline. CP-SAT found proven-optimal solutions in at most 0.11 s on every instance; greedy was within 0.81% and the GA within 1.0% on average on the full instance, at a much greater computational cost. The result that the simplest methods work so well is a genuine finding and not a failure of the experiment: it shows that NP-hardness must be read together with the structure of the instance at hand. The optimum also funds far more projects and captures 27.8% more votes than the city's rank-by-votes selection, with the caveats given in Section 6.4. The natural next step is to add constraints that break the benign structure and see when heuristics start to pay off.